Tarefa 18: Comparando Soma de Vetores entre CPU e GPU em OpenMP

Matheus Bezeril Pinto

Outubro de 2026

Resumo

Este relatório compara a soma de vetores em CPU e em GPU NVIDIA Tesla V100 com OpenMP. Foram analisadas três séries CPU e três séries V100, totalizando 30 medições e três execuções de aquecimento. Cada execução soma dez milhões de elementos em precisão simples. As médias CPU foram de 10,568 ms e 9,975 ms com uma thread configurada, e 5,379 ms com quatro. Na V100, as médias ficaram entre 12,176 ms e 13,484 ms. Todos os registros apresentaram zero erros. A CPU foi mais rápida na operação medida. A interpretação considera a baixa quantidade de cálculo por elemento e os custos de movimentar os dados e iniciar a execução na GPU. Como os logs medem a chamada completa, não se atribui uma parcela numérica do tempo às transferências ou ao kernel isoladamente. O texto explica as diretivas, o intervalo medido e a dispersão dos tempos; os códigos e os registros completos estão nos apêndices.

1 Objetivo e seleção das execuções

O enunciado solicita realizar os exercícios de adição de vetores do tutorial de GPUs com OpenMP no NPAD, comparar CPU e GPU e apresentar os problemas encontrados e as soluções adotadas [1, 2]. A comparação deve considerar o que está dentro da medição: executar uma operação completa com dados inicialmente no host pode ter um resultado diferente de medir apenas um kernel com dados já na GPU.

A implementação está organizada nos arquivos `t18.c`, `aux18.c`, `somas.c` e `jobs.sh` [3]. A análise utiliza as três séries CPU e as três séries com Tesla V100-SXM2-16GB. Foram mantidos o mesmo tamanho dos vetores e a mesma tolerância; as execuções com outros aceleradores não integram a comparação. A identificação dos registros é:

Série	Arquivo original	Configuração
CPU 1a	`CPU1a.out`	CPU, 1 thread
CPU 1b	`CPU1b.out`	CPU, 1 thread
CPU 4	`CPU4.out`	CPU, 4 threads
V100 a	`GPUa.out`	GPU V100
V100 b	`GPUb.out`	GPU V100
V100 c	`resultado_gpu_2134904.out`	GPU V100

Tabela 1: Arquivos utilizados na comparação CPU/V100. As estatísticas são calculadas separadamente para cada série.

O enunciado cita os slides 27 e 48. Na edição SC23 disponível no tutorial NPAD consultado, os exercícios equivalentes aparecem nas páginas 29 e 50 do PDF: primeiro a distribuição do laço na CPU, depois o uso de `target` e `loop` na GPU.

2 Operação e quantidade de trabalho

2.1 Soma elemento a elemento

Dados os vetores a e b, calcula-se

$$c_i = a_i + b_i, \qquad i = 0, \ldots, N-1.$$

Cada resultado depende apenas de duas entradas com o mesmo índice. Não existe uma soma acumulada entre posições. Por isso, diferentes índices podem ser processados simultaneamente, sem que uma iteração precise esperar pelo resultado de outra.

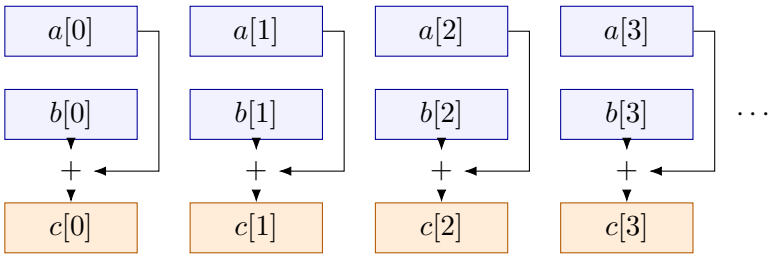


Figura 1: Adição elemento a elemento. A independência entre índices permite distribuir o laço na CPU ou em um dispositivo com offload.

Nesta implementação, $N = 10^7$, os vetores usam `float`, a_i recebe a conversão de i, b_i recebe duas vezes esse valor e c_i começa zerado. O vetor `res` guarda a expressão de referência $a_i + b_i$, também em precisão simples, para a comparação final.

2.2 Intensidade aritmética e volume de dados

Com quatro bytes por `float`, cada vetor ocupa 40 MB. Os quatro vetores no host ocupam 160 MB: duas entradas, uma saída e uma referência. Os dados úteis da soma são

80 MB de entrada e 40 MB de saída.

Cada elemento exige uma adição, duas leituras e uma escrita. A *intensidade aritmética*, isto é, a quantidade de cálculo em relação aos dados acessados, é aproximadamente

$$I = \frac{1 \text{ operação}}{12 \text{ bytes}} \approx 0{,}083 \text{ operação/byte.}$$

Essa relação é pequena: o algoritmo movimenta dados para realizar apenas uma soma por elemento. Portanto, uma grande capacidade de cálculo da GPU não garante uma chamada mais rápida quando é necessário disponibilizar as entradas no dispositivo e devolver o resultado ao host.

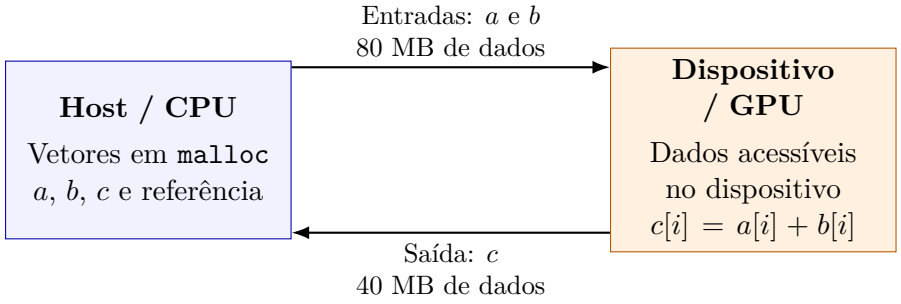


Figura 2: Modelo de movimentação dos dados úteis entre host e dispositivo. O desenho representa as entradas e a saída necessárias à operação, sem medir o tráfego efetivo do runtime.

A figura representa as regiões de dados úteis de uma transferência explícita. Não é uma medição de tráfego. A implementação utiliza ponteiros de `malloc` sem cláusulas `map` explícitas; a forma efetiva de disponibilizar esses dados depende do suporte e do modelo de memória do compilador e do runtime. Os logs não registram bytes transferidos, e os 120 MB não são apresentados como uma medição dessas transferências.

3 Implementação e conceitos de OpenMP

3.1 Organização dos arquivos

`t18.c` recebe tamanho, tolerância e modo, chama as etapas e mede a soma. `aux18.c` contém leitura dos argumentos, alocação, inicialização, validação e liberação. `somas.c` contém as duas funções de soma. `jobs.sh` solicita recursos ao Slurm, compila e executa as repetições.

Os dois arquivos auxiliares são incluídos diretamente em `t18.c`. Portanto, o comando compila somente `t18.c`; adicionar os mesmos arquivos novamente ao comando produziria definições duplicadas. A alocação com `malloc` permite reservar os vetores no heap, evitando depender de espaço suficiente na pilha para quatro arrays grandes. As conversões dos argumentos verificam formato e faixa, e as alocações são verificadas antes do uso.

3.2 CPU: distribuição do laço com parallel for

```
#pragma omp parallel for
for (int i = 0; i < N; i++) {
    c[i] = a[i] + b[i];
}
```

A diretiva cria uma região paralela e distribui as iterações pela equipe de threads. Cada thread escreve em elementos diferentes de c. A sincronização ao final garante que a soma esteja concluída antes da continuação do programa.

O percurso em índices crescentes acessa posições contíguas dos vetores. Isso favorece a localidade espacial: uma linha de cache traz dados próximos que poderão ser usados nas próximas iterações. O compilador também pode vetorizar a soma, processando vários elementos por instrução. Vetorização e distribuição entre threads são mecanismos distintos.

3.3 GPU: target, loop e MANDATORY

```
#pragma omp target
#pragma omp loop
for (int i = 0; i < N; i++) {
    c[i] = a[i] + b[i];
}
```

`target` delimita a região destinada ao dispositivo; `loop` descreve o laço que pode ser distribuído no contexto de execução disponível. O encaminhamento da execução do host para o dispositivo é chamado *offload*. Nesta tarefa, o host inicia o programa e realiza a validação, enquanto o modo GPU utiliza a região target para a soma [5].

O job GPU contém explicitamente:

```
export OMP_TARGET_OFFLOAD=MANDATORY
```

No padrão OpenMP, `MANDATORY` exige a execução no dispositivo para essas regiões target e determina falha caso o offload não possa ser realizado, em vez de permitir uma execução silenciosa no host, chamada *fallback*. Essa política foi definida no job das execuções V100. Seu funcionamento pressupõe compilador e runtime compatíveis com offload e com as diretivas utilizadas.

A variável define a política de execução. Ela não divide automaticamente o vetor entre GPUs, não especifica o mapeamento dos vetores e não mantém os dados disponíveis entre processos distintos. Assim, exigir o uso da GPU não elimina os custos que entram na chamada medida.

3.4 Validação e redução

Depois da soma, o programa conta as posições em que

$$|c_i - \mathrm{res}_i| > 10^{-9}.$$

A contagem usa `parallel for reduction(+:err)`: cada thread acumula sua contribuição, e as contribuições são somadas ao final. Isso evita atualizações concorrentes desprotegidas no mesmo contador.

Todos os 33 registros selecionados, incluindo os aquecimentos, mostraram zero erros pelo critério implementado. A referência usa a mesma expressão em `float`; esse resultado verifica a soma nesse critério, sem significar precisão decimal de nove casas. A validação fica fora do intervalo cronometrado.

4 Ambiente e metodologia

4.1 Recursos do NPAD

Os logs CPU identificam `service3.npad.internal`, com uma ou quatro threads configuradas. Os logs V100 identificam `gpunode1.npad.internal` e Tesla V100-SXM2-16GB. A documentação de hardware do NPAD informa que a classe `gpu-8-v100` possui, por nó, dois Intel Xeon E5-2683 v4, 512 GB DDR4 e oito GPUs V100 com 16 GB por GPU [4]. RAM do host e memória da GPU são recursos distintos. O modelo da CPU de `service3` não consta dos registros do experimento.

O script CPU solicita `intel-128`, um nó e uma tarefa. O script apresentado configura uma CPU por tarefa; o log CPU 4 documenta a variação com quatro threads. A quantidade configurada é exportada para `OMP_NUM_THREADS`. As variáveis `OMP_PLACES=cores` e `OMP_PROC_BIND=close` orientam o posicionamento das threads nos locais de execução.

O script GPU solicita a partição `gpu-8-v100` e quatro GPUs. A quantidade de GPUs existentes no nó ou reservadas ao job não equivale à quantidade usada pelo algoritmo: o fonte possui uma região target e não implementa uma divisão do vetor entre dispositivos.

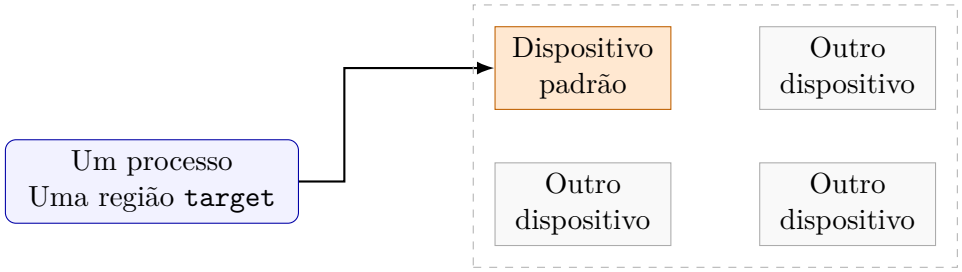


Figura 3: Oito GPUs no nó e quatro GPUs solicitadas ao Slurm não significam uma soma distribuída por quatro dispositivos. O programa possui uma região target, sem particionamento explícito do vetor entre GPUs.

Os dois modos utilizam o comando de compilação apresentado no job, com `-O3`, OpenMP e C11:

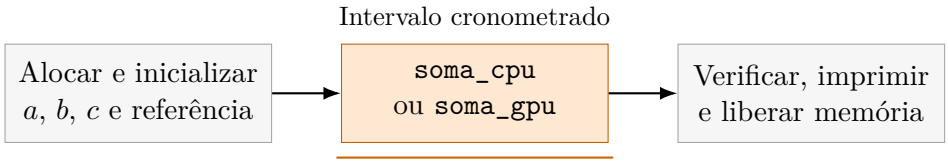
```
gcc -O3 -fopenmp -std=c11 t18.c -o executavel -lm
```

O nome do executável e o argumento final selecionam CPU ou GPU. As comparações

preservam $N = 10^7$ e tolerância 10^{-9}; não se misturam resultados de outras classes de aceleradores.

4.2 Intervalo de medição

`omp_get_wtime()` é chamado antes e depois de `soma_cpu` ou `soma_gpu`. O número impresso é o tempo da chamada completa. Alocação, inicialização, construção da referência, verificação e impressão ficam fora desse intervalo.



Em offload com transferência explícita, a chamada inclui envio, execução e retorno dos dados. Os logs não separam esses custos.

Figura 4: O tempo impresso mede a chamada completa de soma. Não inclui inicialização nem validação; a região target é síncrona no código apresentado.

A região target não usa `nowait`; a chamada espera sua conclusão. Assim, o tempo GPU é avaliado como o custo da operação a partir dos dados no host, incluindo os custos de offload presentes no caminho executado. Não se trata de uma medição isolada do kernel com os dados previamente residentes na GPU.

4.3 Repetições, aquecimento e CV

São seis séries de cinco medições: 30 tempos utilizados nas médias. As três séries V100 possuem uma execução anterior identificada como `Warm-up`, excluída da média, totalizando 33 registros preservados.

Cada chamada ao executável inicia um processo e realiza uma soma. Por isso, o aquecimento anterior não garante reutilizar o mesmo contexto de GPU no processo seguinte. As cinco medições continuam sujeitas aos custos iniciais presentes em cada chamada. Isso caracteriza o experimento, em vez de uma avaliação de um contexto persistente já aquecido.

Para cinco tempos T_r, usam-se média $\overline{T}$, desvio padrão amostral s e coeficiente de variação:

$$\overline{T} = \frac{1}{5}\sum_{r=1}^{5} T_r, \qquad s = \sqrt{\frac{\sum_{r=1}^{5}(T_r - \overline{T})^2}{4}}, \qquad CV = 100\frac{s}{\overline{T}}.$$

O *coeficiente de variação* (CV) expressa a dispersão dos tempos em relação à média. Por exemplo, média de 10 ms e desvio de 0,2 ms correspondem a CV de 2%. Um CV baixo indica tempos próximos entre si; não significa menor tempo, menor erro numérico ou maior eficiência da GPU. As barras do gráfico representam um desvio padrão amostral.

5 Resultados CPU e V100

5.1 Tempos médios e dispersão

Tabela 2: Séries CPU e Tesla V100. Cinco medições por série; aquecimentos separados e excluídos das médias.

Série	Média (ms)	s (ms)	CV (%)	Aquecimento (ms)
CPU 1a	10,567663	0,019582	0,19	—
CPU 1b	9,974812	0,038587	0,39	—
CPU 4	5,379129	0,037736	0,70	—
V100 a	12,927817	0,127933	0,99	13,064327
V100 b	13,484205	0,339940	2,52	14,371583
V100 c	12,175676	0,339729	2,79	12,091021

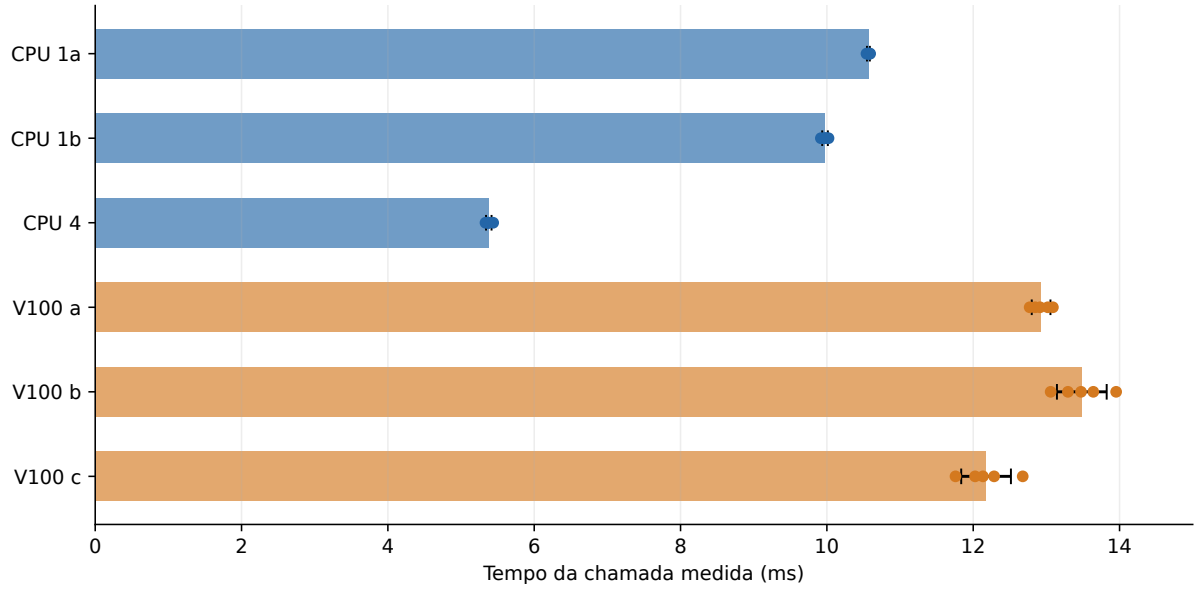


Figura 5: As 30 medições selecionadas, com médias e desvios amostrais. Azul: CPU. Laranja: V100. Os três aquecimentos GPU não entram no gráfico nem nas médias.

A CPU com quatro threads apresentou a menor média, 5,379 ms. As duas séries CPU de uma thread também foram mais rápidas que todas as séries V100. Os CVs variam de aproximadamente 0,19% a 2,79%, indicando pequena dispersão interna nas séries.

As médias CPU 1a e CPU 1b diferem cerca de 5,94%, mais que seus desvios internos. Portanto, as duas referências são mantidas separadas. O mesmo cuidado é aplicado às três séries V100: não se escolhe apenas a mais rápida para representar todo o experimento.

5.2 Ganho com quatro threads na CPU

O speedup relativo e a eficiência com quatro threads são

$$S_4 = \frac{\overline{T}_1}{\overline{T}_4}, \qquad E_4 = \frac{S_4}{4}.$$

Tabela 3: Comparação CPU com o mesmo N. As duas referências permanecem separadas.

Referência de uma thread	T_1/T_4	E_4 (%)	T_4 (ms)
CPU 1a	1,9646	49,11	5,379129
CPU 1b	1,8544	46,36	5,379129

A aceleração fica entre 1,854 e 1,965, conforme a referência de uma thread, e a eficiência entre 46,36% e 49,11%. Isso corresponde a um ganho próximo de duas vezes ao usar quatro threads. Eficiência aqui é a aproximação ao ganho ideal de quatro vezes, não a porcentagem de utilização da CPU.

O acesso contíguo favorece caches e vetorização, mas várias threads compartilham recursos de memória e precisam ser coordenadas. Como há pouco cálculo por elemento, o ganho não precisa acompanhar proporcionalmente o número de threads. Os logs não separam os custos desses mecanismos.

5.3 Comparação da chamada GPU com a CPU

Tabela 4: Razão entre tempos médios. Valores maiores que um indicam maior tempo da chamada GPU em relação à referência CPU da coluna.

Série	$T_{GPU}/T_{CPU\,1a}$	$T_{GPU}/T_{CPU\,1b}$	$T_{GPU}/T_{CPU\,4}$
V100 a	1,223	1,296	2,403
V100 b	1,276	1,352	2,507
V100 c	1,152	1,221	2,264

As médias V100 ficaram entre 12,176 ms e 13,484 ms. Em relação à CPU com quatro threads, a chamada GPU demorou aproximadamente 2,26 a 2,51 vezes mais. A vantagem CPU também aparece nas duas referências de uma thread, embora seja menor.

Essa comparação responde à operação medida: somar os vetores a partir da organização de dados do programa. Ela não demonstra que uma soma com os dados já disponíveis na V100 seria mais lenta que o laço CPU, pois esse cenário não foi medido separadamente.

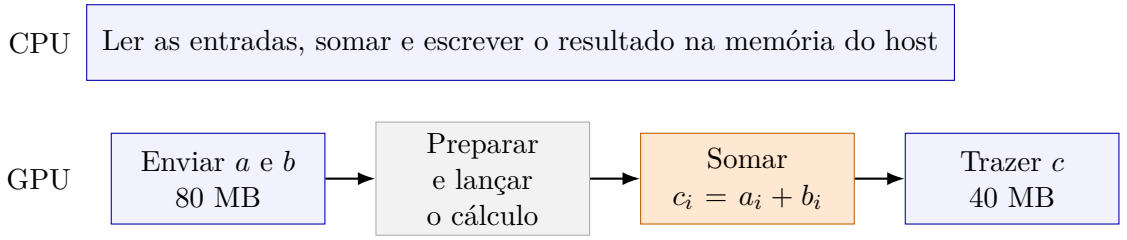
6 Discussão dos custos de execução

6.1 Tempo total e tempo do kernel

O *kernel* é a parte do cálculo executada no dispositivo. Para uma operação com dados inicialmente no host e transferência explícita, um modelo conceitual do tempo é

$$T_{GPU} = T_{\text{envio}} + T_{\text{preparação}} + T_{\text{kernel}} + T_{\text{retorno}} + T_{\text{sincronização}}.$$

Na CPU, os vetores já estão na memória do host. Também existe custo para lê-los e escrever a saída, mas não é necessário disponibilizá-los em outro dispositivo e trazer o resultado de volta. Uma GPU pode executar o kernel rapidamente e, ainda assim, perder na soma desses custos.



O tempo total da chamada depende de todas as etapas.
Uma soma por elemento oferece pouco cálculo para amortizar a movimentação.

Figura 6: Custos de uma soma com dados inicialmente no host e transferência explícita. Os blocos não estão em escala de tempo e não representam uma decomposição medida. O envio e o retorno ocorrem na entrada e na saída da região, não uma vez por iteração do laço.

A interpretação adotada para o resultado é que a movimentação dos dados e os custos de offload não foram compensados pelo pouco cálculo realizado. Cada elemento participa de apenas uma soma. O potencial de paralelismo da GPU não elimina o tempo necessário para disponibilizar as entradas, iniciar a execução e retornar a saída.

A movimentação considerada ocorre por regiões dos vetores, sem pressupor uma cópia a cada iteração de `for`. Para a soma completa, são duas regiões de entrada e uma de saída, representadas pelos 120 MB de dados úteis. Em cada novo processo, o programa realiza novamente sua operação, sem reutilizar dados de uma chamada anterior. O custo é grande em relação ao trabalho matemático, mesmo sem uma cópia por elemento.

6.2 Alcance da análise

Os tempos mostram a vantagem CPU para essa soma e essa forma de medição. A comparação envolve nós distintos e se restringe às configurações testadas. A baixa intensidade aritmética oferece uma explicação coerente para o peso da movimentação. Entretanto, os logs não medem envio, kernel e retorno separadamente. Portanto, não se atribui uma porcentagem específica às transferências nem se calcula uma largura de banda a partir de um tempo que inclui outras etapas.

Um algoritmo que realize mais operações por elemento ou aproveite os dados em vários cálculos oferece mais trabalho para amortizar a movimentação. Apenas aumentar N mantém a mesma proporção de uma soma por elemento, embora possa reduzir a importância relativa de custos fixos de lançamento.

7 Progresso e decisões de implementação

Questão	Decisão adotada	Motivo
Vetores grandes	Alocação com `malloc` e teste de falha	Reservar memória sem depender de grandes arrays na pilha.
Entradas inválidas	Conversão com checagem de formato e faixa	Evitar tamanhos, tolerâncias e modos inválidos.
Laço CPU	`parallel for`	Distribuir índices independentes entre threads.
Modo GPU	`target`, `loop` e política `MANDATORY` no job	Destinar o cálculo ao dispositivo e exigir offload na configuração OpenMP.
Verificação	Referência e `reduction(+:err)`	Comparar a saída e combinar as contagens sem disputa no contador.
Comparação de tempos	Mesmos vetores e intervalo em torno da soma	Comparar a operação completa, explicitando seus custos.

Tabela 5: Principais decisões da implementação e suas finalidades.

8 Conclusão

Na soma de dez milhões de elementos, a CPU apresentou menor tempo que a V100 em todas as séries selecionadas. Com quatro threads, a média CPU foi de 5,379 ms; na V100, ficou entre 12,176 ms e 13,484 ms. Todos os registros apresentaram zero erros pelo verificador implementado.

O resultado é interpretado a partir do custo da operação completa e da baixa intensidade aritmética: movimentar os dados e iniciar a execução na GPU não foi compensado por apenas uma soma por elemento. A configuração `MANDATORY` define a exigência de offload, mas não elimina esses custos. A medição não separa as parcelas para quantificá-las individualmente.

Os resultados mostram que a escolha do recurso de execução depende da quantidade de cálculo por dado e do intervalo efetivamente medido. Para este trabalho simples, a CPU foi a opção mais rápida; a capacidade de cálculo da GPU, sozinha, não determina o desempenho da aplicação completa.

A Enunciado e proveniência

A.1 Requisitos completos

```
Nome da Tarefa:
Tarefa 18: Comparando soma de vetores entre CPU e GPU em OpenMP
Descrição:

Faça os exercícios de adição de vetores, vadd.c, dos slides 27 e 48 do tutorial de programação de
    GPUs com OpenMP em um dos nós com GPU do NPAD. Compare os tempos de execução somente na CPU e
    com o uso da GPU. Reporte seu progresso apresentando os problemas encontrados e as soluções
    apresentadas.

Tutorial sobre programação de GPUs com OpenMP da Universidade de Bristol:

https://github.com/UoB-HPC/openmp-tutorial

https://github.com/NPAD-UFRN/openmp-tutorial (adaptação para o NPAD)
```

A.2 Seleção e proveniência dos arquivos

```
Fontes e registros originais: commit e17b7d07f4e0ae1a3f1cdd65e0a8f587638e0bbf.
Conjunto de dados utilizado na analise:
  CPU1a.out, CPU1b.out e CPU4.out.
  GPUa.out, GPUb.out e resultado_gpu_2134904.out: Tesla V100-SXM2-16GB.
Outras classes de GPU: fora do escopo da comparacao.
Total: 30 medicoes e tres aquecimentos (33 registros).
Implementacao: t18.c, aux18.c, somas.c e jobs.sh.
O job GPU exporta OMP_TARGET_OFFLOAD=MANDATORY.
As series CPU de uma e quatro threads sao identificadas pelos logs.
Datas exatas e versoes do compilador de cada serie nao estao registradas.
Tutorial NPAD consultado: a8ceec45357cd71594f569aed98ed631e9461a8b.
Os logs medem a chamada completa, sem separar transferencia e kernel.
```

A.3 Referência de hardware da classe V100

```
Fonte: https://npad.ufrn.br/npad/hardware
Especificacoes de referencia da classe V100 em 18/10/2026.

Classe GPU Tesla V100 / particao gpu-8-v100:
  2 nos; por no:
  2 x Intel Xeon E5-2683 v4
  512 GB DDR4 (memoria do host)
  8 x NVIDIA Tesla V100-SXM2 de 16 GB (memoria por GPU)
  SSD local de 1,92 TB
  InfiniBand ConnectX-4 FDR 56 Gb/s

Logs V100 selecionados: hostname gpunode1.npad.internal.
O job solicita quatro GPUs; o codigo nao divide a soma entre dispositivos.
Os logs CPU identificam service3.npad.internal, com uma ou quatro threads.
O modelo de CPU de service3 nao foi registrado.
```

B Códigos completos

Os códigos e o script utilizados são apresentados integralmente a seguir. A inclusão usa `\lstinputlisting`, sem limites de linhas, para manter os apêndices sincronizados com os arquivos do projeto.

B.1 Programa principal: t18.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

#include "aux18.c"
#include "somas.c"

int main(int argc, char *argv[])
{
    if (argc != 4) {
        fprintf(stderr, "Uso: %s <N> <TOL> <MODO>\n", argv[0]);
        fprintf(stderr, "  N    = tamanho dos vetores\n");
        fprintf(stderr, "  TOL  = tolerancia\n");
        fprintf(stderr, "  MODO = 0 (CPU) ou 1 (GPU)\n");

        return EXIT_FAILURE;
    }

    // Leitura e validacao dos argumentos
    int N = ler_n(argv[1]);
    float TOL = ler_tol(argv[2]);
    int modo = ler_modo(argv[3]);

    // Alocacao
    float *a = aloca_vetor(N);
    float *b = aloca_vetor(N);
    float *c = aloca_vetor(N);
    float *res = aloca_vetor(N);

    // Inicializacao
    inicializa_vetores(a, b, c, res, N);

    // Execucao
    double inicio = omp_get_wtime();

    if (modo == 0) {
        soma_cpu(a, b, c, N);
    } else {
        soma_gpu(a, b, c, N);
    }

    double fim = omp_get_wtime();

    // Validacao
    int err = verifica_resultado(c, res, N, TOL);

    // Resultado
    printf("====================================\n");
    printf("Soma de vetores\n");
    printf("N      : %d\n", N);
    printf("TOL    : %e\n", TOL);
    printf("MODO   : %s\n", modo == 0 ? "CPU" : "GPU");
    printf("Erros  : %d\n", err);
    printf("Tempo  : %.9f s\n", fim - inicio);
    printf("====================================\n");

    // Liberacao
```

```
58      libera_vetores(a, b, c, res);
59
60      return EXIT_SUCCESS;
61  }
```

B.2 Somas CPU e GPU: somas.c

```c
#include <stdlib.h>
#include <omp.h>

void soma_cpu(float *a, float *b, float *c, int N)
{
    #pragma omp parallel for
    for (int i = 0; i < N; i++) {
        c[i] = a[i] + b[i];
    }
}

void soma_gpu(float *a, float *b, float *c, int N)
{
    #pragma omp target
    #pragma omp loop
    for (int i = 0; i < N; i++) {
        c[i] = a[i] + b[i];
    }
}
```

B.3 Funções auxiliares: aux18.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <errno.h>
#include <limits.h>
#include <math.h>

int ler_n(const char *arg)
{
    char *end;
    long valor;

    errno = 0;
    valor = strtol(arg, &end, 10);

    if (end == arg || *end != '\0') {
        fprintf(stderr, "Erro: N invalido: '%s'\n", arg);
        exit(EXIT_FAILURE);
    }

    if (errno == ERANGE || valor > INT_MAX || valor < INT_MIN) {
        fprintf(stderr, "Erro: N fora do intervalo permitido: '%s'\n", arg);
        exit(EXIT_FAILURE);
    }

    if (valor <= 0) {
        fprintf(stderr, "Erro: N deve ser maior que zero.\n");
        exit(EXIT_FAILURE);
    }

    return (int)valor;
}


float ler_tol(const char *arg)
{
    char *end;
    float valor;

    errno = 0;
    valor = strtof(arg, &end);

    if (end == arg || *end != '\0') {
```

```
43          fprintf(stderr, "Erro: TOL invalida: '%s'\n", arg);
44          exit(EXIT_FAILURE);
45      }
46
47      if (errno == ERANGE) {
48          fprintf(stderr, "Erro: TOL fora do intervalo permitido: '%s'\n", arg);
49          exit(EXIT_FAILURE);
50      }
51
52      if (!isfinite(valor) || valor < 0.0f) {
53          fprintf(stderr, "Erro: TOL deve ser um valor finito maior ou igual a zero.\n");
54          exit(EXIT_FAILURE);
55      }
56
57      return valor;
58  }
59
60
61  int ler_modo(const char *arg)
62  {
63      char *end;
64      long valor;
65
66      errno = 0;
67      valor = strtol(arg, &end, 10);
68
69      if (end == arg || *end != '\0') {
70          fprintf(stderr, "Erro: MODO invalido: '%s'\n", arg);
71          exit(EXIT_FAILURE);
72      }
73
74      if (errno == ERANGE || valor < INT_MIN || valor > INT_MAX) {
75          fprintf(stderr, "Erro: MODO invalido: '%s'\n", arg);
76          exit(EXIT_FAILURE);
77      }
78
79      if (valor != 0 && valor != 1) {
80          fprintf(stderr, "Erro: MODO deve ser 0 (CPU) ou 1 (GPU).\n");
81          exit(EXIT_FAILURE);
82      }
83
84      return (int)valor;
85  }
86
87
88  float *aloca_vetor(int N)
89  {
90      float *vetor = malloc((size_t)N * sizeof(float));
91
92      if (vetor == NULL) {
93          fprintf(stderr, "Erro: falha na alocacao de memoria.\n");
94          exit(EXIT_FAILURE);
95      }
96
97      return vetor;
98  }
99
100
101 void inicializa_vetores(float *a, float *b, float *c, float *res, int N)
102 {
103     for (int i = 0; i < N; i++) {
104         a[i] = (float)i;
105         b[i] = 2.0f * (float)i;
106         c[i] = 0.0f;
107         res[i] = a[i] + b[i];
108     }
109 }
110
111
112 int verifica_resultado(float *c, float *res, int N, float TOL)
```

```
113 {
114     int err = 0;
115 
116     #pragma omp parallel for reduction(+:err)
117     for (int i = 0; i < N; i++) {
118         float diff = c[i] - res[i];
119 
120         if (fabsf(diff) > TOL) {
121             err++;
122         }
123     }
124 
125     return err;
126 }
127 
128 
129 void libera_vetores(float *a, float *b, float *c, float *res)
130 {
131     free(a);
132     free(b);
133     free(c);
134     free(res);
135 }
```

B.4 Submissão: jobs.sh

```
#!/bin/bash

# ============================================================
# Job CPU
# ============================================================

sbatch <<'EOF'
#!/bin/bash

#SBATCH --partition=intel-128
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=1
#SBATCH --time=00:02:00
#SBATCH --job-name=ppCPU_job
#SBATCH --output=resultado_cpu_%j.out

cd "$SLURM_SUBMIT_DIR"

gcc -O3 -fopenmp -std=c11 t18.c -o t18_cpu -lm

export OMP_NUM_THREADS=$SLURM_CPUS_PER_TASK
export OMP_PLACES=cores
export OMP_PROC_BIND=close

echo "======================================"
echo "EXECUCAO CPU"
echo "Hostname: $(hostname)"
echo "Threads OpenMP: $OMP_NUM_THREADS"
echo "======================================"
echo

for i in {1..5}; do
    echo "----- Execucao $i -----"
    ./t18_cpu 10000000 1e-9 0
    echo
done
EOF


# ============================================================
# Job GPU
# ============================================================

sbatch <<'EOF'
#!/bin/bash

#SBATCH --partition=gpu-8-v100
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --gres=gpu:4
#SBATCH --gres-flags=enforce-binding
#SBATCH --time=00:02:00
#SBATCH --job-name=ppGPU_job
#SBATCH --output=resultado_gpu_%j.out

cd "$SLURM_SUBMIT_DIR"

export OMP_TARGET_OFFLOAD=MANDATORY

gcc -O3 -fopenmp -std=c11 t18.c -o t18_gpu -lm

echo "======================================"
echo "EXECUCAO GPU"
echo "Hostname: $(hostname)"
echo "======================================"

```

```
nvidia-smi --query-gpu=name --format=csv,noheader
echo

echo "----- Warm-up -----"
./t18_gpu 10000000 1e-9 1
echo

for i in {1..5}; do
    echo "----- Execucao $i -----"
    ./t18_gpu 10000000 1e-9 1
    echo
done
EOF
```

C Medições completas e identificação dos arquivos

Os aquecimentos são as repetições zero. Todos os valores pertencem às séries CPU e V100 selecionadas; as médias utilizam somente as cinco repetições medidas de cada série.

Tabela 6: Todos os 33 registros selecionados. Repetição 0 identifica o aquecimento em processo separado.

Série	Repetição	Tempo (s)	Erros
CPU 1a	1	0,010591173	0
CPU 1a	2	0,010564717	0
CPU 1a	3	0,010541467	0
CPU 1a	4	0,010558852	0
CPU 1a	5	0,010582105	0
CPU 1b	1	0,009982172	0
CPU 1b	2	0,010022212	0
CPU 1b	3	0,009961779	0
CPU 1b	4	0,009917792	0
CPU 1b	5	0,009990105	0
CPU 4	1	0,005374323	0
CPU 4	2	0,005372279	0
CPU 4	3	0,005439168	0
CPU 4	4	0,005334313	0
CPU 4	5	0,005375561	0
V100 a	0	0,013064327	0
V100 a	1	0,013018875	0
V100 a	2	0,012913231	0
V100 a	3	0,013088888	0
V100 a	4	0,012771590	0
V100 a	5	0,012846501	0
V100 b	0	0,014371583	0
V100 b	1	0,013473431	0
V100 b	2	0,013641344	0
V100 b	3	0,013953403	0
V100 b	4	0,013058480	0
V100 b	5	0,013294365	0
V100 c	0	0,012091021	0
V100 c	1	0,012676509	0
V100 c	2	0,012025466	0
V100 c	3	0,012285496	0
V100 c	4	0,011758101	0
V100 c	5	0,012132809	0

C.1 Hashes SHA-256 dos fontes e registros selecionados

```
e5d343c02fdd9cbb33270056119140b679a5c9451010a71d49f19f43969f8bbc  t18.c
7de66807492db57081c4c1eb79bc61d50750108f57a40d658989851e1000ae11  somas.c
b29a394a8574e3cde627484c2eaa9062f86d2686f340260746c593f16b597876  aux18.c
e74a5034251b1b880a672a3b1d9ed083cd67670d6d01ab19a6f9cf771d5d2ce9  jobs.sh
```

```
20f27067c758831623b8d2aa12fab74885f446e1abbe698331c44afc9d43b6a5  requisitosTarefa18.txt
dafcab4a64266d3d45630b2d53a4cbc8ceb445909604353d82016291359d0ada  CPU1a.out
fa7d39550a34cc2209fc7fde3249da3dfdc8ac4defceb3022c9a85a8006f9387  CPU1b.out
d41b8c82eb0c5be135c565331cab434dcb976847b1a7e4f5fa257e55dc643d21  CPU4.out
e8347f8056edac440eebc7c9bcf6067527baadfa874729b32768e4bc38acabda  GPUa.out
ab0ded7015290506cf4c34204c342ea24cfd7473aac274ba6df65848d793458f  GPUb.out
89757b8df5ca845c27b529a6f40f12a1a9f2f5294e82ebaf706991b759a30d01  resultado_gpu_2134904.out
```

D Registros integrais das execuções selecionadas

D.1 CPU1a.out

```
====================================
EXECUCAO CPU
Hostname: service3.npad.internal
Threads OpenMP: 1
====================================

----- Execucao 1 -----
==================================
Soma de vetores
N      : 10000000
TOL    : 1.000000e-09
MODO   : CPU
Erros  : 0
Tempo  : 0.010591173 s
==================================

----- Execucao 2 -----
==================================
Soma de vetores
N      : 10000000
TOL    : 1.000000e-09
MODO   : CPU
Erros  : 0
Tempo  : 0.010564717 s
==================================

----- Execucao 3 -----
==================================
Soma de vetores
N      : 10000000
TOL    : 1.000000e-09
MODO   : CPU
Erros  : 0
Tempo  : 0.010541467 s
==================================

----- Execucao 4 -----
==================================
Soma de vetores
N      : 10000000
TOL    : 1.000000e-09
MODO   : CPU
Erros  : 0
Tempo  : 0.010558852 s
==================================

----- Execucao 5 -----
==================================
Soma de vetores
N      : 10000000
TOL    : 1.000000e-09
MODO   : CPU
Erros  : 0
Tempo  : 0.010582105 s
==================================
```

D.2 CPU1b.out

```
====================================
EXECUCAO CPU
Hostname: service3.npad.internal
Threads OpenMP: 1
====================================

----- Execucao 1 -----
==================================
Soma de vetores
N      : 10000000
TOL    : 1.000000e-09
MODO   : CPU
Erros  : 0
Tempo  : 0.009982172 s
==================================

----- Execucao 2 -----
==================================
Soma de vetores
N      : 10000000
TOL    : 1.000000e-09
MODO   : CPU
Erros  : 0
Tempo  : 0.010022212 s
==================================

----- Execucao 3 -----
==================================
Soma de vetores
N      : 10000000
TOL    : 1.000000e-09
MODO   : CPU
Erros  : 0
Tempo  : 0.009961779 s
==================================

----- Execucao 4 -----
==================================
Soma de vetores
N      : 10000000
TOL    : 1.000000e-09
MODO   : CPU
Erros  : 0
Tempo  : 0.009917792 s
==================================

----- Execucao 5 -----
==================================
Soma de vetores
N      : 10000000
TOL    : 1.000000e-09
MODO   : CPU
Erros  : 0
Tempo  : 0.009990105 s
==================================
```

D.3 CPU4.out

```
====================================
EXECUCAO CPU
Hostname: service3.npad.internal
Threads OpenMP: 4
====================================

----- Execucao 1 -----
==================================
Soma de vetores
N      : 10000000
TOL    : 1.000000e-09
MODO   : CPU
Erros  : 0
Tempo  : 0.005374323 s
==================================

----- Execucao 2 -----
==================================
Soma de vetores
N      : 10000000
TOL    : 1.000000e-09
MODO   : CPU
Erros  : 0
Tempo  : 0.005372279 s
==================================

----- Execucao 3 -----
==================================
Soma de vetores
N      : 10000000
TOL    : 1.000000e-09
MODO   : CPU
Erros  : 0
Tempo  : 0.005439168 s
==================================

----- Execucao 4 -----
==================================
Soma de vetores
N      : 10000000
TOL    : 1.000000e-09
MODO   : CPU
Erros  : 0
Tempo  : 0.005334313 s
==================================

----- Execucao 5 -----
==================================
Soma de vetores
N      : 10000000
TOL    : 1.000000e-09
MODO   : CPU
Erros  : 0
Tempo  : 0.005375561 s
==================================
```

D.4 GPUa.out: V100 a

```
====================================
EXECUCAO GPU
Hostname: gpunode1.npad.internal
====================================
Tesla V100-SXM2-16GB

----- Warm-up -----
==================================
Soma de vetores
N      : 10000000
TOL    : 1.000000e-09
MODO   : GPU
Erros  : 0
Tempo  : 0.013064327 s
==================================

----- Execucao 1 -----
==================================
Soma de vetores
N      : 10000000
TOL    : 1.000000e-09
MODO   : GPU
Erros  : 0
Tempo  : 0.013018875 s
==================================

----- Execucao 2 -----
==================================
Soma de vetores
N      : 10000000
TOL    : 1.000000e-09
MODO   : GPU
Erros  : 0
Tempo  : 0.012913231 s
==================================

----- Execucao 3 -----
==================================
Soma de vetores
N      : 10000000
TOL    : 1.000000e-09
MODO   : GPU
Erros  : 0
Tempo  : 0.013088888 s
==================================

----- Execucao 4 -----
==================================
Soma de vetores
N      : 10000000
TOL    : 1.000000e-09
MODO   : GPU
Erros  : 0
Tempo  : 0.012771590 s
==================================

----- Execucao 5 -----
==================================
Soma de vetores
N      : 10000000
TOL    : 1.000000e-09
MODO   : GPU
Erros  : 0
Tempo  : 0.012846501 s
==================================
```

D.5 GPUb.out: V100 b

```
====================================
EXECUCAO GPU
Hostname: gpunode1.npad.internal
====================================
Tesla V100-SXM2-16GB

----- Warm-up -----
==================================
Soma de vetores
N      : 10000000
TOL    : 1.000000e-09
MODO   : GPU
Erros  : 0
Tempo  : 0.014371583 s
==================================

----- Execucao 1 -----
==================================
Soma de vetores
N      : 10000000
TOL    : 1.000000e-09
MODO   : GPU
Erros  : 0
Tempo  : 0.013473431 s
==================================

----- Execucao 2 -----
==================================
Soma de vetores
N      : 10000000
TOL    : 1.000000e-09
MODO   : GPU
Erros  : 0
Tempo  : 0.013641344 s
==================================

----- Execucao 3 -----
==================================
Soma de vetores
N      : 10000000
TOL    : 1.000000e-09
MODO   : GPU
Erros  : 0
Tempo  : 0.013953403 s
==================================

----- Execucao 4 -----
==================================
Soma de vetores
N      : 10000000
TOL    : 1.000000e-09
MODO   : GPU
Erros  : 0
Tempo  : 0.013058480 s
==================================

----- Execucao 5 -----
==================================
Soma de vetores
N      : 10000000
TOL    : 1.000000e-09
MODO   : GPU
Erros  : 0
Tempo  : 0.013294365 s
==================================
```

D.6 resultado_gpu_2134904.out: V100 c

```
====================================
EXECUCAO GPU
Hostname: gpunode1.npad.internal
====================================
Tesla V100-SXM2-16GB
Tesla V100-SXM2-16GB
Tesla V100-SXM2-16GB
Tesla V100-SXM2-16GB

----- Warm-up -----
==================================
Soma de vetores
N      : 10000000
TOL    : 1.000000e-09
MODO   : GPU
Erros  : 0
Tempo  : 0.012091021 s
==================================

----- Execucao 1 -----
==================================
Soma de vetores
N      : 10000000
TOL    : 1.000000e-09
MODO   : GPU
Erros  : 0
Tempo  : 0.012676509 s
==================================

----- Execucao 2 -----
==================================
Soma de vetores
N      : 10000000
TOL    : 1.000000e-09
MODO   : GPU
Erros  : 0
Tempo  : 0.012025466 s
==================================

----- Execucao 3 -----
==================================
Soma de vetores
N      : 10000000
TOL    : 1.000000e-09
MODO   : GPU
Erros  : 0
Tempo  : 0.012285496 s
==================================

----- Execucao 4 -----
==================================
Soma de vetores
N      : 10000000
TOL    : 1.000000e-09
MODO   : GPU
Erros  : 0
Tempo  : 0.011758101 s
==================================

----- Execucao 5 -----
==================================
Soma de vetores
N      : 10000000
TOL    : 1.000000e-09
MODO   : GPU
Erros  : 0
Tempo  : 0.012132809 s
```

```
==================================
```

E Código completo de geração do gráfico e das tabelas

O script utiliza uma lista explícita de arquivos selecionados e verifica a identificação V100 nos logs GPU. Ele extrai os 33 registros, calcula as estatísticas das 30 medições e gera os artefatos do relatório. Não executa novas simulações.

```
"""Analisa somente as séries CPU e Tesla V100."""
from pathlib import Path
import csv
import re
from statistics import mean, stdev

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

ROOT = Path(__file__).resolve().parent
DATA, FIG, TABLE = (ROOT / name for name in ("dados", "figuras", "tabelas"))
FIG.mkdir(exist_ok=True)
TABLE.mkdir(exist_ok=True)
SERIES = (
    ("CPU1a.out", "CPU 1a", "CPU", 1, "service3", "CPU não registrada"),
    ("CPU1b.out", "CPU 1b", "CPU", 1, "service3", "CPU não registrada"),
    ("CPU4.out", "CPU 4", "CPU", 4, "service3", "CPU não registrada"),
    ("GPUa.out", "V100 a", "GPU", None, "gpunode1", "Tesla V100-SXM2-16GB"),
    ("GPUb.out", "V100 b", "GPU", None, "gpunode1", "Tesla V100-SXM2-16GB"),
    ("resultado_gpu_2134904.out", "V100 c", "GPU", None, "gpunode1",
     "Tesla V100-SXM2-16GB"),
)
records, summaries = [], []
for filename, label, mode, threads, host, model in SERIES:
    text = (DATA / "logs" / filename).read_text(encoding="utf-8")
    assert f"Hostname: {host}.npad.internal" in text
    if mode == "GPU":
        assert "Tesla V100-SXM2-16GB" in text
    chunks = re.split(r"----- (Warm-up|Execucao \d+) -----", text)
    times, warmup = [], None
    for i in range(1, len(chunks), 2):
        stage, block = chunks[i:i + 2]
        n = int(re.search(r"N\s*:\s*(\d+)", block)[1])
        tol = float(re.search(r"TOL\s*:\s*([\deE+.-]+)", block)[1])
        declared = re.search(r"MODO\s*:\s*(CPU|GPU)", block)[1]
        errors = int(re.search(r"Erros\s*:\s*(\d+)", block)[1])
        elapsed = float(re.search(r"Tempo\s*:\s*([\d.]+)", block)[1])
        assert n == 10000000 and tol == 1e-9 and declared == mode and errors == 0
        repetition = 0 if stage == "Warm-up" else int(stage.split()[-1])
        records.append((filename, label, mode, threads, host, repetition,
                        n, tol, errors, elapsed))
        if repetition == 0:
            warmup = elapsed
        else:
            times.append(elapsed)
    assert len(times) == 5
    avg, sd = mean(times), stdev(times)
    summaries.append(dict(file=filename, label=label, mode=mode, threads=threads,
                          host=host, model=model, warmup=warmup, times=times,
                          avg=avg, sd=sd, cv=100 * sd / avg))
assert len(records) == 33

with (DATA / "tempos_extraidos.csv").open("w", newline="", encoding="utf-8") as f:
    writer = csv.writer(f, delimiter=";", lineterminator="\n")
    writer.writerow(("arquivo", "serie", "modo", "threads_configuradas",
                     "hostname", "repeticao_0_aquecimento", "N", "tolerancia",
                     "erros", "tempo_segundos"))
    writer.writerows(records)
with (DATA / "resumo.csv").open("w", newline="", encoding="utf-8") as f:
    writer = csv.writer(f, delimiter=";", lineterminator="\n")
    writer.writerow(("arquivo", "serie", "media_segundos", "desvio_amostral",
```

```
                      "cv_percentual", "aquecimento_segundos"))
    for row in summaries:
        writer.writerow((row["file"], row["label"], row["avg"], row["sd"],
                         row["cv"], row["warmup"]))


def pt(number, digits=3):
    return f"{number:.{digits}f}".replace(".", ",")


def table(name, columns, heading, caption, rows, label):
    text = [r"\begin{longtable}{" + columns + "}", r"\caption{" + caption +
            r"}\label{" + label + r"}\\", r"\toprule " + heading + r"\\",
            r"\midrule\endfirsthead", r"\toprule " + heading + r"\\",
            r"\midrule\endhead", r"\bottomrule\endfoot"]
    text.extend(" & ".join(row) + r"\\" for row in rows)
    text.append(r"\end{longtable}")
    (TABLE / name).write_text("\n".join(text) + "\n", encoding="utf-8")


table("resumo.tex", "lrrrr",
      r"Série & Média (ms) & $s$ (ms) & CV (\%) & Aquecimento (ms)",
      "Séries CPU e Tesla V100. Cinco medições por série; "
      "aquecimentos separados e excluídos das médias.",
      [[row["label"], pt(row["avg"] * 1000, 6), pt(row["sd"] * 1000, 6),
        pt(row["cv"], 2), "---" if row["warmup"] is None else pt(row["warmup"] * 1000, 6)]
       for row in summaries], "tab:resumo")
table("brutos.tex", "lrrr",
      r"Série & Repetição & Tempo (s) & Erros",
      "Todos os 33 registros selecionados. Repetição 0 identifica o aquecimento "
      "em processo separado.",
      [[row[1], str(row[5]), pt(row[9], 9), str(row[8])] for row in records], "tab:brutos")
table("cpu.tex", "lrrr",
      r"Referência de uma thread & $T_1/T_4$ & $E_4$ (\%) & $T_4$ (ms)",
      "Comparação CPU com o mesmo N. As duas referências permanecem separadas.",
      [[row["label"], pt(row["avg"] / summaries[2]["avg"], 4),
        pt(100 * row["avg"] / summaries[2]["avg"] / 4, 2),
        pt(summaries[2]["avg"] * 1000, 6)] for row in summaries[:2]], "tab:cpu")
table("comparacao.tex", "lrrr",
      r"Série & $T_{GPU}/T_{CPU\,1a}$ & $T_{GPU}/T_{CPU\,1b}$ & $T_{GPU}/T_{CPU\,4}$",
      "Razão entre tempos médios. Valores maiores que um indicam maior tempo "
      "da chamada GPU em relação à referência CPU da coluna.",
      [[row["label"], *[pt(row["avg"] / cpu["avg"], 3) for cpu in summaries[:3]]]
       for row in summaries[3:]], "tab:comparacao")

plt.rcParams.update({"font.size": 10, "axes.spines.top": False,
                     "axes.spines.right": False, "pdf.fonttype": 42})
fig, ax = plt.subplots(figsize=(8.5, 4.2), layout="constrained")
for i, row in enumerate(summaries):
    color = "#2366a8" if row["mode"] == "CPU" else "#d4791f"
    ax.barh(i, row["avg"] * 1000, color=color, alpha=.65, height=.58)
    ax.errorbar(row["avg"] * 1000, i, xerr=row["sd"] * 1000,
                fmt="none", ecolor="black", capsize=4)
    ax.scatter([value * 1000 for value in row["times"]], [i] * 5,
               color=color, s=25, zorder=3)
ax.set(yticks=range(len(summaries)), yticklabels=[row["label"] for row in summaries],
       xlabel="Tempo da chamada medida (ms)", xlim=(0, 15))
ax.invert_yaxis()
ax.grid(axis="x", alpha=.22)
fig.savefig(FIG / "tempos.pdf", bbox_inches="tight")
plt.close(fig)
print("Seis séries: 30 medições e três aquecimentos CPU/V100 conferidos.")
```

Referências

[1] Material da disciplina de Programação Paralela. *Tarefa 18: Comparando soma de vetores entre CPU e GPU em OpenMP*. Enunciado reproduzido no Apêndice A.

[2] UNIVERSITY OF BRISTOL; NPAD/UFRN. *Programming Your GPU with OpenMP*. Tutorial e adaptação NPAD. Versão NPAD consultada: `a8ceec4`. Disponível em: `https://github.com/UoB-HPC/openmp-tutorial` e `https://github.com/NPAD-UFRN/openmp-tutorial`.

[3] PINTO, Matheus Bezeril. *ProgParalela2026.2: Tarefa 18*. Fontes e registros do experimento disponíveis no commit `e17b7d0`. Disponível em: `https://github.com/39Matheus/ProgParalela2026.2/tree/e17b7d0/Tarefa_18`.

[4] NPAD/UFRN. *Hardware*. Especificações de referência da classe V100. Referência de 18 de outubro de 2026, reproduzida no Apêndice A. Disponível em: `https://npad.ufrn.br/npad/hardware`.

[5] OPENMP ARCHITECTURE REVIEW BOARD. *OpenMP Application Programming Interface*. Conceitos de target, loop, reduction e política de offload. Disponível em: `https://www.openmp.org/specifications/`.